

Resumo das questoes 18, 19, 20 e orientacao da questao 21

arquivo de testes das questoes:

src/main/java/com/mycompany/hamburgueria_poo_raulbeltrame_matheusdias/Testes
QuestoesSistema.java

Um main separado só pra demonstrar Iterator, foreach, Comparator e
Collections.sort sem mexer no funcionamento normal

alterado para a q20:

src/main/java/com/mycompany/hamburgueria_poo_raulbeltrame_matheusdias/Cadast
roClientes.java

Essa q20 a alteração foi implementada no sistema real e testada na classe de testes

Questao 18

Instanciar um iterator para a ArrayList de pessoas, funcionario ou cliente, percorrer com while(iterator.hasNext()) e imprimir iterator.next(). Explicar como acontece, relacionar com foreach e testar foreach.

```
q18 percorrendo com Iterator
Funcionario{Usuario{idUsuario=1, nome='Raul', login='raul'}}
Funcionario{Usuario{idUsuario=2, nome='Matheus', login='matheus'}}
Funcionario{Usuario{idUsuario=3, nome='Atendente', login='atendente'}}

q18 percorrendo com foreach
Funcionario{Usuario{idUsuario=1, nome='Raul', login='raul'}}
Funcionario{Usuario{idUsuario=2, nome='Matheus', login='matheus'}}
Funcionario{Usuario{idUsuario=3, nome='Atendente', login='atendente'}}
```

o que fizemos:

No TestesQuestoesSistema.java crieio bloco:

```
// questao 18
```

dentro dele eu criei um CadastroUsuarios, cadastrei 3 funcionarios e copieei a lista para uma ArrayList.

depois:

```
Iterator<Funcionario> iterator = funcionarios.iterator();
```

e percorro a lista :

```
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

depois o mesmo teste usando foreach:

```
for (Funcionario funcionario : funcionarios) {  
    System.out.println(funcionario);  
}
```

explicação:

O Iterator é como um cursor que passa pela lista. O método hasNext() pergunta se ainda existe um próximo elemento. O método next() devolve esse elemento e avança o cursor para a próxima posição.

O foreach em Java é uma forma mais curta de fazer essa mesma ideia. Quando usamos:

```
for (Funcionario funcionario : funcionarios)
```

o Java usa o mecanismo de Iterable/Iterator da coleção. Por isso o foreach é uma forma simplificada e mais legível de percorrer a lista.

as duas partes imprimem os mesmos funcionários.

Questao 19

Apresentar no main testes do comparador implementado. Usar Collections.sort passando o comparador criado para ordenar a lista com dois parametros diferentes. Rodar duas vezes.

Já tínhamos a classe Cliente e já implementava Comparator. O método compare da classe Cliente compara pelo idCliente:

```
public int compare(Cliente primeiroCliente, Cliente segundoCliente) {  
    return Integer.compare(primeiroCliente.getIdCliente(),  
        segundoCliente.getIdCliente());  
}
```

A classe Cliente também possui um comparador por nome:

```
public static Comparator<Cliente> porNome()
```

O que fiz:

No arquivo TestesQuestoesSistema.java criei o bloco:

```
// questao 19
```

Dentro dele foi criada um ArrayList fora de ordem

Primeiro foi mostrada a lista original

Depois fiz a ordenacao por idCliente:

```
Collections.sort(clientes, new Cliente());
```

Esse comando usa o Comparator implementado na propria classe Cliente

Depois outra ordenacao por nome

```
Collections.sort(clientes, Cliente.porNome());
```

Esse comando usa outro Comparator criado na classe Cliente.

Entao, Collections.sort(lista, comparator) ordena a lista original usando o criterio definido no comparator. O comparator recebe dois objetos e decide qual deve vir antes

Na primeira ordenacao, o criterio e o idCliente

Na segunda ordenacao, o criterio e o nome.

q19 lista original

```
Cliente{idCliente=3, nome='Bruno', telefone='3333-3333', endereco='Rua C', idUltimoPedido=103, dataUltimoPedido=2026-06-03, quantidadePedidos=0, quantidadeRecibos=0}  
Cliente{idCliente=1, nome='Carlos', telefone='1111-1111', endereco='Rua A', idUltimoPedido=101, dataUltimoPedido=2026-06-01, quantidadePedidos=0, quantidadeRecibos=0}  
Cliente{idCliente=2, nome='Ana', telefone='2222-2222', endereco='Rua B', idUltimoPedido=102, dataUltimoPedido=2026-06-02, quantidadePedidos=0, quantidadeRecibos=0}
```

q19 Collections.sort por idCliente usando Comparator implementado em Cliente

```
Cliente{idCliente=1, nome='Carlos', telefone='1111-1111', endereco='Rua A', idUltimoPedido=101, dataUltimoPedido=2026-06-01, quantidadePedidos=0, quantidadeRecibos=0}  
Cliente{idCliente=2, nome='Ana', telefone='2222-2222', endereco='Rua B', idUltimoPedido=102, dataUltimoPedido=2026-06-02, quantidadePedidos=0, quantidadeRecibos=0}  
Cliente{idCliente=3, nome='Bruno', telefone='3333-3333', endereco='Rua C', idUltimoPedido=103, dataUltimoPedido=2026-06-03, quantidadePedidos=0, quantidadeRecibos=0}
```

q19 Collections.sort por nome usando outro Comparator da classe Cliente

```
Cliente{idCliente=2, nome='Ana', telefone='2222-2222', endereco='Rua B', idUltimoPedido=102, dataUltimoPedido=2026-06-02, quantidadePedidos=0, quantidadeRecibos=0}  
Cliente{idCliente=3, nome='Bruno', telefone='3333-3333', endereco='Rua C', idUltimoPedido=103, dataUltimoPedido=2026-06-03, quantidadePedidos=0, quantidadeRecibos=0}  
Cliente{idCliente=1, nome='Carlos', telefone='1111-1111', endereco='Rua A', idUltimoPedido=101, dataUltimoPedido=2026-06-01, quantidadePedidos=0, quantidadeRecibos=0}
```

Na saida, a lista original aparece fora de ordem. Depois ela aparece ordenada por id e depois aparece ordenada por nome

Questao 20

Criar um metodo find para clientes utilizando Iterator e Comparator. Apresentar testes do metodo implementado. Fazer chamadas ao binarySearch() da classe Collections e comparar com o find implementado.

O que fiz:

No CadastroClientes.java foi criado o metodo:

```
public Cliente find(Cliente clienteProcurado, Comparator<Cliente>  
comparator) {  
    Iterator<Cliente> iterator = getClientes().iterator();  
    while (iterator.hasNext()) {  
        Cliente cliente = iterator.next();  
        if (comparator.compare(cliente, clienteProcurado) == 0) {  
            return cliente;  
        }  
    }  
}
```

```

        }
    }
    return null;
}

```

Esse metodo percorre a lista com Iterator e usa o Comparator recebido para saber se o cliente atual e igual ao cliente procurado

No TestesQuestoesSistema.java foi criado o bloco:

```
// questao 20
```

Foram feitos dois testes

Teste 1: busca por idCliente.

Criamos:

```

Comparator<Cliente> comparadorPorId = new Cliente();
Cliente clienteProcuradoPorId = new Cliente(2, "", "", "", 0, null);

```

Depois chamei:

```
cadastroClientes.find(clienteProcuradoPorId, comparadorPorId);
```

Depois a lista foi ordenada com o mesmo comparator:

```
Collections.sort(clientes, comparadorPorId);
```

E foi chamado:

```
Collections.binarySearch(clientes, clienteProcuradoPorId, comparadorPorId);
```

Teste 2: busca por nome.

Foi criado:

```

Comparator<Cliente> comparadorPorNome = Cliente.porNome();
Cliente clienteProcuradoPorNome = new Cliente(0, "Bruno", "", "", 0, null);

```

Depois chamamos find com esse comparator.

Depois a lista foi ordenada por nome e foi chamado binarySearch com o mesmo comparator.

Então, O find implementado faz busca linear. Ele percorre do inicio ao fim usando Iterator. Ele nao precisa que a lista esteja ordenada.

O binarySearch faz busca binaria. Ele e mais rapido em listas grandes, mas so funciona corretamente se a lista estiver ordenada pelo mesmo Comparator usado na busca.

Outra diferenca importante:

O find retorna o Cliente encontrado ou null.

O `binarySearch` retorna o índice onde o Cliente foi encontrado. Se não encontrar, retorna um número negativo.

q20 find por `idCliente` usando `Iterator` e `Comparator`

`Cliente{idCliente=2, nome='Ana', telefone='2222-2222', endereco='Rua B', idUltimoPedido=102, dataUltimoPedido=2026-06-02, quantidadePedidos=0, quantidadeRecibos=0}`

q20 `binarySearch` por `idCliente` após ordenar com o mesmo `Comparator`

índice: 1

`Cliente{idCliente=2, nome='Ana', telefone='2222-2222', endereco='Rua B', idUltimoPedido=102, dataUltimoPedido=2026-06-02, quantidadePedidos=0, quantidadeRecibos=0}`

q20 find por nome usando `Iterator` e `Comparator`

`Cliente{idCliente=3, nome='Bruno', telefone='3333-3333', endereco='Rua C', idUltimoPedido=103, dataUltimoPedido=2026-06-03, quantidadePedidos=0, quantidadeRecibos=0}`

q20 `binarySearch` por nome após ordenar com o mesmo `Comparator`

índice: 1

`Cliente{idCliente=3, nome='Bruno', telefone='3333-3333', endereco='Rua C', idUltimoPedido=103, dataUltimoPedido=2026-06-03, quantidadePedidos=0, quantidadeRecibos=0}`

Nos testes feitos, o `find` e o `binarySearch` encontram o mesmo cliente

Sistema principal

Main:

`com.mycompany.hamburgueria_poo_raulbeltrame_matheusdias.Hamburgueria_poo_raulBeltrame_matheusDias`

Login de adm fixo

login: Raul senha: 6504

configurado no método `ConfigurarAdministradorPrincipal`

Questão 21

Apresentar o funcionamento básico para o atendimento de 10 clientes da hamburgueria, com variações em cada processo, desde cadastro do cliente até criação dos pedidos, baixas no estoque, testes de entrega ou desvio, cadastro das vendas, finalizando com emissão de nota fiscal ou cancelamento da venda após a conclusão dos processos.

O que o nosso sistema tem hoje

O sistema tem cadastro de clientes, funcionários, produtos, ingredientes, estoque, pedidos, vendas, recibo e entrega.

A venda gera Recibo. Não temos uma classe chamada `NotaFiscal`. Então o recibo gerado pelo sistema está sendo usado como comprovante final da venda

O cancelamento implementado no sistema é cancelamento de pedido antes da venda. A opção está no menu do funcionário como “cancelar pedido”. Não existe hoje um método separado tipo `cancelarVenda` depois que a venda já foi registrada.

Sequencia Questao 21 (JSON limpo para testes)

Cadastro um funcionario

```
opcao: 2
nome do funcionario: Vava
login: 123
senha: 123
funcionario incluido: Funcionario{Usuario{idUsuario=6, nome='Vava', login='123'}}
```

Cadastro cliente

```
opcao: 6
Cliente{idCliente=2, nome='João Victor Fernandes', telefone='(38) 99812-4501', endereco='Rua da Glória, 125, Centro, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=3, nome='Maria Eduarda Silva', telefone='(38) 99734-5821', endereco='Rua Direita, 87, Centro, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=4, nome='Gabriel Henrique Souza', telefone='(38) 99845-1236', endereco='Gabriel', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=5, nome='Ana Clara Oliveira', telefone='(38) 99678-3452', endereco='Rua das Mercês, 59, Bairro Rio Grande, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=6, nome='Pedro Lucas Martins', telefone='(38) 99791-6784', endereco='Rua do Amparo, 301, Centro, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=7, nome='Júlia Fernandes Costa', telefone='(38) 99823-4567', endereco='Rua da Quitanda, 142, Centro, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=8, nome='Rafael Gomes Pereira', telefone='(38) 99654-7891', endereco='Rua do Bonfim, 78, Bairro Bom Jesus, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=9, nome='Larissa Almeida Rocha', telefone='(38) 99867-2345', endereco='Rua Caminho dos Escravos, 190, Bairro Palha, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=10, nome='Matheus Rodrigues Lima', telefone='(38) 99745-8912', endereco='Rua São Francisco, 223, Bairro Bela Vista, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
Cliente{idCliente=11, nome='Camila Vitória Santos', telefone='(38) 99890-5678', endereco='Rua da Caridade, 110, Bairro Fátima, Diamantina - MG', idUltimoPedido=0, dataUltimoPedido=null, quantidadePedidos=0, quantidadeRecibos=0}
```

Cadastrar motoqueiro

```
opcao: 17
nome do motoqueiro: Carlos Eduardo Batista

telefone: valor obrigatorio
telefone: (38) 99712-6345

motoqueiro ativo? (s/n): digite s ou n
motoqueiro ativo? (s/n): s
motoqueiro cadastrado: Motoqueiro{idMotoqueiro=1, nome='Carlos Eduardo Batista', telefone='(38) 99712-6345', ativo=true}

opcao: 17
nome do motoqueiro: Tiago Henrique Mendes

telefone: valor obrigatorio
telefone:
(38) 99683-7451
valor obrigatorio
telefone: motoqueiro ativo? (s/n): s
motoqueiro cadastrado: Motoqueiro{idMotoqueiro=2, nome='Tiago Henrique Mendes', telefone='(38) 99683-7451', ativo=true}
```

Cadastrar regiões de entrega

```
opcao: 18
nome da regioao: Central
bairros atendidos: Centro, Bom Jesus
regiao cadastrada: RegiaoEntrega{idRegiao=1, nomeRegiao='Central', bairros='Centro, Bom Jesus'}

opcao: 18
nome da regioao: Rio Grande
bairros atendidos: Rio Grande, Palha
regiao cadastrada: RegiaoEntrega{idRegiao=2, nomeRegiao='Rio Grande', bairros='Rio Grande, Palha'}

opcao: 18
nome da regioao: Bela Vista
bairros atendidos: Bela Vista
regiao cadastrada: RegiaoEntrega{idRegiao=3, nomeRegiao='Bela Vista', bairros='Bela Vista'}
```

Cadastrar itens no estoque com muita quantidade para criar 10 pedidos e mostrar estoque

```
opcao: 14
Ingrediente{idIngrediente=1, nome='Pão brioche', quantidadeAtual=300.0, unidadeMedida='2', quantidadeMinima=50.0}
Ingrediente{idIngrediente=2, nome='hambúrguer 120g', quantidadeAtual=250.0, unidadeMedida='1', quantidadeMinima=85.0}
Ingrediente{idIngrediente=3, nome='queijo mussarela', quantidadeAtual=200.0, unidadeMedida='1', quantidadeMinima=50.0}
Ingrediente{idIngrediente=4, nome='alface', quantidadeAtual=150.0, unidadeMedida='1', quantidadeMinima=50.0}
Ingrediente{idIngrediente=5, nome='tomate', quantidadeAtual=150.0, unidadeMedida='0.5', quantidadeMinima=40.0}
Ingrediente{idIngrediente=6, nome='milho', quantidadeAtual=150.0, unidadeMedida='0.5', quantidadeMinima=25.0}
Ingrediente{idIngrediente=7, nome='queijo cheddar', quantidadeAtual=150.0, unidadeMedida='1', quantidadeMinima=25.0}
Ingrediente{idIngrediente=8, nome='bacon', quantidadeAtual=250.0, unidadeMedida='3', quantidadeMinima=65.0}
Ingrediente{idIngrediente=9, nome='ovo', quantidadeAtual=300.0, unidadeMedida='1', quantidadeMinima=60.0}
Ingrediente{idIngrediente=10, nome='cheddar', quantidadeAtual=150.0, unidadeMedida='2', quantidadeMinima=60.0}
Ingrediente{idIngrediente=11, nome='cebola caramelizada', quantidadeAtual=250.0, unidadeMedida='1', quantidadeMinima=25.0}
```

Cadastrar produtos com os itens

```
opcao: 10
Produto{idProduto=1, nome='X-Burger Clássico', ativo=true, descricaoProduto=DescricaoProduto{idDescricao=1, descricaoDetalhada='Hambúrguer bo'}
Produto{idProduto=2, nome='X-Salada', ativo=true, descricaoProduto=DescricaoProduto{idDescricao=2, descricaoDetalhada='X-Salada Especial', va'}
Produto{idProduto=3, nome='X-Bacon', ativo=true, descricaoProduto=DescricaoProduto{idDescricao=3, descricaoDetalhada='Hambúrguer bovino, ched'}
Produto{idProduto=4, nome='X-Egg Bacon', ativo=true, descricaoProduto=DescricaoProduto{idDescricao=4, descricaoDetalhada='Hambúrguer bovino 1'}
Produto{idProduto=5, nome='Duplo Cheddar', ativo=true, descricaoProduto=DescricaoProduto{idDescricao=5, descricaoDetalhada='Dois hambúrgueres'}}
```


Criar pedidos para cada um dos 10 e printar variações da etapa da entrega

```
opcao: 12
Pedido{idPedido=1, dataPedido=2026-06-14, horarioPedido=18:56:00, horarioEntregaPrevisto=20:00:00, status=SAIU_PARA_ENTREGA, valorTotal=53.8}
Pedido{idPedido=2, dataPedido=2026-06-14, horarioPedido=19:15:00, horarioEntregaPrevisto=20:10:00, status=ENTREGUE, valorTotal=56.4, valorTotal=56.4}
Pedido{idPedido=3, dataPedido=2026-06-14, horarioPedido=19:25:00, horarioEntregaPrevisto=20:35:00, status=SAIU_PARA_ENTREGA, valorTotal=53.8}
Pedido{idPedido=4, dataPedido=2026-06-14, horarioPedido=19:45:00, horarioEntregaPrevisto=20:55:00, status=EM_PREPARO, valorTotal=36.0, valorTotal=36.0}
Pedido{idPedido=5, dataPedido=2026-06-14, horarioPedido=20:01:00, horarioEntregaPrevisto=21:05:00, status=CANCELADO, valorTotal=111.0, valorTotal=111.0}
Pedido{idPedido=6, dataPedido=2026-06-14, horarioPedido=20:04:00, horarioEntregaPrevisto=21:05:00, status=EM_PREPARO, valorTotal=70.6999, valorTotal=70.6999}
Pedido{idPedido=7, dataPedido=2026-06-14, horarioPedido=20:26:00, horarioEntregaPrevisto=21:26:00, status=RECEBIDO, valorTotal=63.9, valorTotal=63.9}
Pedido{idPedido=8, dataPedido=2026-06-14, horarioPedido=20:30:00, horarioEntregaPrevisto=21:31:00, status=RECEBIDO, valorTotal=18.0, valorTotal=18.0}
Pedido{idPedido=9, dataPedido=2026-06-14, horarioPedido=20:33:00, horarioEntregaPrevisto=21:40:00, status=ENTREGUE, valorTotal=46.8, valorTotal=46.8}
Pedido{idPedido=10, dataPedido=2026-06-14, horarioPedido=20:40:00, horarioEntregaPrevisto=21:40:00, status=SAIU_PARA_ENTREGA, valorTotal=53.8}
```

Mostrar baixa no estoque após fechar os ciclos dos 10 pedidos

```
opcao: 14
Ingrediente{idIngrediente=1, nome='Pão brioche', quantidadeAtual=264.0, unidadeMedida='2', quantidadeMinima=50.0}
Ingrediente{idIngrediente=2, nome='hambúrguer 120g', quantidadeAtual=228.0, unidadeMedida='1', quantidadeMinima=85.0}
Ingrediente{idIngrediente=3, nome='queijo mussarela', quantidadeAtual=197.0, unidadeMedida='1', quantidadeMinima=50.0}
Ingrediente{idIngrediente=4, nome='alface', quantidadeAtual=130.5, unidadeMedida='1', quantidadeMinima=50.0}
Ingrediente{idIngrediente=5, nome='tomate', quantidadeAtual=135.5, unidadeMedida='0.5', quantidadeMinima=40.0}
Ingrediente{idIngrediente=6, nome='milho', quantidadeAtual=142.5, unidadeMedida='0.5', quantidadeMinima=25.0}
Ingrediente{idIngrediente=7, nome='queijo cheddar', quantidadeAtual=150.0, unidadeMedida='1', quantidadeMinima=25.0}
Ingrediente{idIngrediente=8, nome='bacon', quantidadeAtual=232.0, unidadeMedida='3', quantidadeMinima=65.0}
Ingrediente{idIngrediente=9, nome='ovo', quantidadeAtual=296.0, unidadeMedida='1', quantidadeMinima=60.0}
Ingrediente{idIngrediente=10, nome='cheddar', quantidadeAtual=122.0, unidadeMedida='2', quantidadeMinima=60.0}
Ingrediente{idIngrediente=11, nome='cebola caramelizada', quantidadeAtual=246.0, unidadeMedida='1', quantidadeMinima=25.0}
```

Mostrar recibos dos pedidos

```
opcao: 9
id do pedido: 1
venda registrada: Venda{idVenda=1, dataVenda=2026-06-14, valorTotal=53.8, pedido=Pedido{idPedido=1, dataPedido=2026-06-14, horarioPedido=18:56:00, horarioEntregaPrevisto=20:00:00, status=EM_PREPARO}, valorTotal=53.8}
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 1
Cliente 1
Valor total 53.8', entrega=null}
recibo gerado: Recibo{idRecibo=1, dataEmissao=2026-06-14, detalhes='Loja Hmmmmburguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 1
Cliente 1
Valor total 53.8'}
```

```
opcao: 9
id do pedido: 2
venda registrada: Venda{idVenda=2, dataVenda=2026-06-14, valorTotal=56.4, pedido=Pedido{idPedido=2, dataPedido=2026-06-14, horarioPedido=19:15:00, horarioEntregaPrevisto=20:10:00, status=ENTREGUE}, valorTotal=56.4}
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 2
Cliente 2
Valor total 56.4', entrega=null}
recibo gerado: Recibo{idRecibo=2, dataEmissao=2026-06-14, detalhes='Loja Hmmmmburguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 2
Cliente 2
Valor total 56.4'}
```



```
opcao: 9
id do pedido: 3
venda registrada: Venda{idVenda=3, dataVenda=2026-06-14, valorTotal=71.8, pedido=Pedido{idPedido=3,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 3
Cliente 3
Valor total 71.8'}, entrega=null}
recibo gerado: Recibo{idRecibo=3, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 3
Cliente 3
Valor total 71.8'}

opcao: 9
id do pedido: 4
venda registrada: Venda{idVenda=4, dataVenda=2026-06-14, valorTotal=36.0, pedido=Pedido{idPedido=4,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 4
Cliente 4
Valor total 36.0'}, entrega=null}
recibo gerado: Recibo{idRecibo=4, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 4
Cliente 4
Valor total 36.0'}
```

```
opcao: 9
id do pedido: 5
venda registrada: Venda{idVenda=5, dataVenda=2026-06-14, valorTotal=111.0, pedido=Pedido{idPedido=5,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 5
Cliente 5
Valor total 111.0'}, entrega=null}
recibo gerado: Recibo{idRecibo=5, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 5
Cliente 5
Valor total 111.0'}
```

opcao: 9
id do pedido: 6
venda registrada: Venda{idVenda=6, dataVenda=2026-06-14, valorTotal=70.69999999999999,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 6
Cliente 6
Valor total 70.69999999999999'}, entrega=null}
recibo gerado: Recibo{idRecibo=6, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 6
Cliente 6
Valor total 70.69999999999999'}

opcao: 9
id do pedido: 7
venda registrada: Venda{idVenda=7, dataVenda=2026-06-14, valorTotal=63.9, pedido=Pedido{idPedido=7,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 7
Cliente 7
Valor total 63.9'}, entrega=null}
recibo gerado: Recibo{idRecibo=7, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 7
Cliente 7
Valor total 63.9'}

opcao: 9
id do pedido: 8
venda registrada: Venda{idVenda=8, dataVenda=2026-06-14, valorTotal=18.0, pedido=Pedido{idPedido=8,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 8
Cliente 8
Valor total 18.0'}, entrega=null}
recibo gerado: Recibo{idRecibo=8, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 8
Cliente 8
Valor total 18.0'}

opcao: 9
id do pedido: 9
venda registrada: Venda{idVenda=7, dataVenda=2026-06-14, valorTotal=46.8, pedido=Pedido{idPedido=9,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 9
Cliente 9
Valor total 46.8'}, entrega=null}
recibo gerado: Recibo{idRecibo=7, dataEmissao=2026-06-14, detalhes='Loja Hmmmbuguer
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 9
Cliente 9
Valor total 46.8'}

```
opcao: 9
id do pedido: 10
venda registrada: Venda{idVenda=8, dataVenda=2026-06-14, valorTotal=44.9, pedido=Pedido{idPedido=10,
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 10
Cliente 10
Valor total 44.9'}, entrega=null}
recibo gerado: Recibo{idRecibo=8, dataEmissao=2026-06-14, detalhes='Loja HmmmBurger
Telefone 38988889999
Endereco Rua X, 10, Jardim
Pedido 10
Cliente 10
Valor total 44.9'}
```

Cancelar alguns pedidos, avançar outros até informar whatsapp (“s” ENTREGUE, “n” SAIU PRA ENTREGA)

```
opcao: 10
id do pedido vendido: 2
Motoqueiro informou entrega Carlos Eduardo Batista
confirmar entrega agora? (s/n): s
entrega atualizada: Entrega{idEntrega=2, statusEntrega='Concluida', confirmacaoWhatsapp=true, motoqueiro=Motoqueiro{idMotoqueiro=1, nome='Carlos Eduardo Batista'}
```

```
opcao: 10
id do pedido vendido: 3
Motoqueiro informou entrega Carlos Eduardo Batista
confirmar entrega agora? (s/n): n
entrega atualizada: Entrega{idEntrega=3, statusEntrega='em rota', confirmacaoWhatsapp=true, motoqueiro=Motoqueiro{idMotoqueiro=1, nome='Carlos Eduardo Batista',
```

```
opcao: 7
id do pedido: 5
pedido cancelado. retencao: 35.349999999999994
```

```
opcao: 10
id do pedido vendido: 10
Motoqueiro informou entrega Carlos Eduardo Batista
confirmar entrega agora? (s/n): n
entrega atualizada: Entrega{idEntrega=5, statusEntrega='em rota', confirmacaoWhatsapp=true, motoqueiro=Motoqueiro{idMotoqueiro=1, nome='Carlos Eduardo Batista'}
```

```
opcao: 10
id do pedido vendido: 9
Motoqueiro informou entrega Carlos Eduardo Batista
confirmar entrega agora? (s/n): s
entrega atualizada: Entrega{idEntrega=4, statusEntrega='Concluida', confirmacaoWhatsapp=true, motoqueiro=Motoqueiro{idMotoqueiro=1, nome='Carlos Eduardo Batista',
```